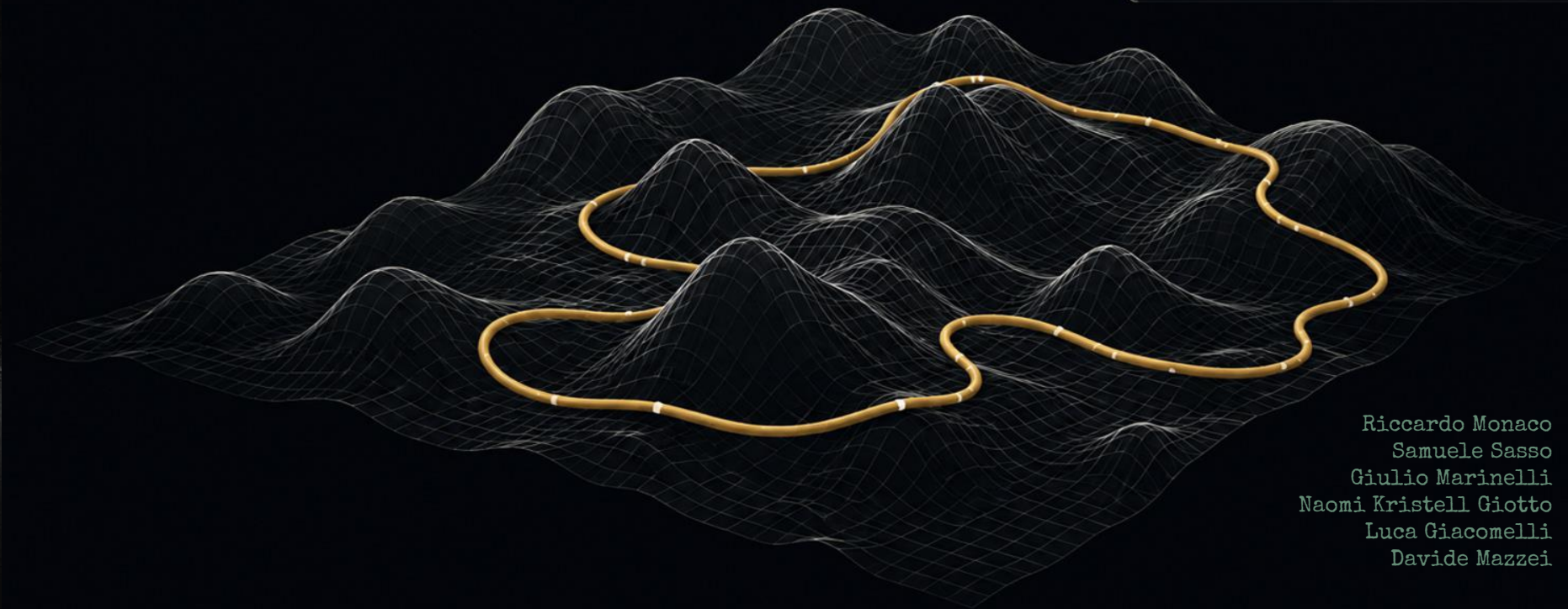
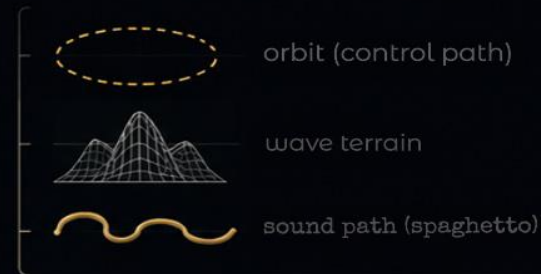


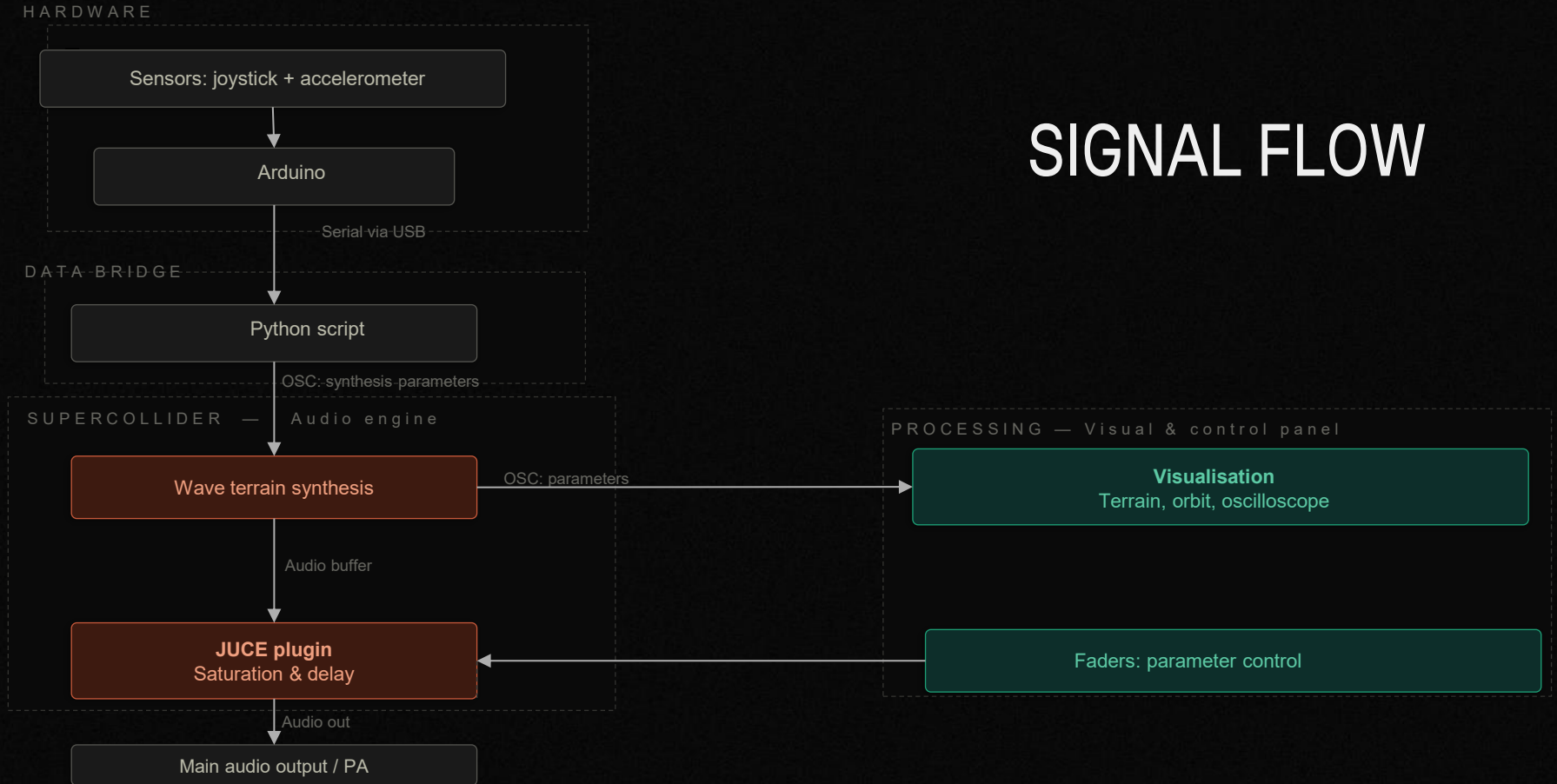
Spaghetti

//wave terrain synthesizer



Riccardo Monaco
Samuele Sasso
Giulio Marinelli
Naomi Kristell Giotto
Luca Giacomelli
Davide Mazzei

SIGNAL FLOW



Gravity: Triple Axis Accelerometer MMA7361



Gravity: Joystick Module



Python Bridge

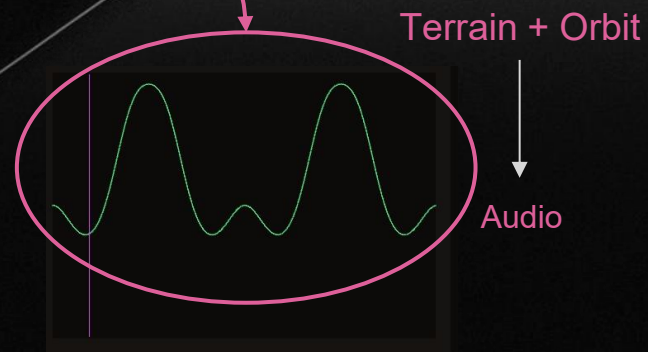
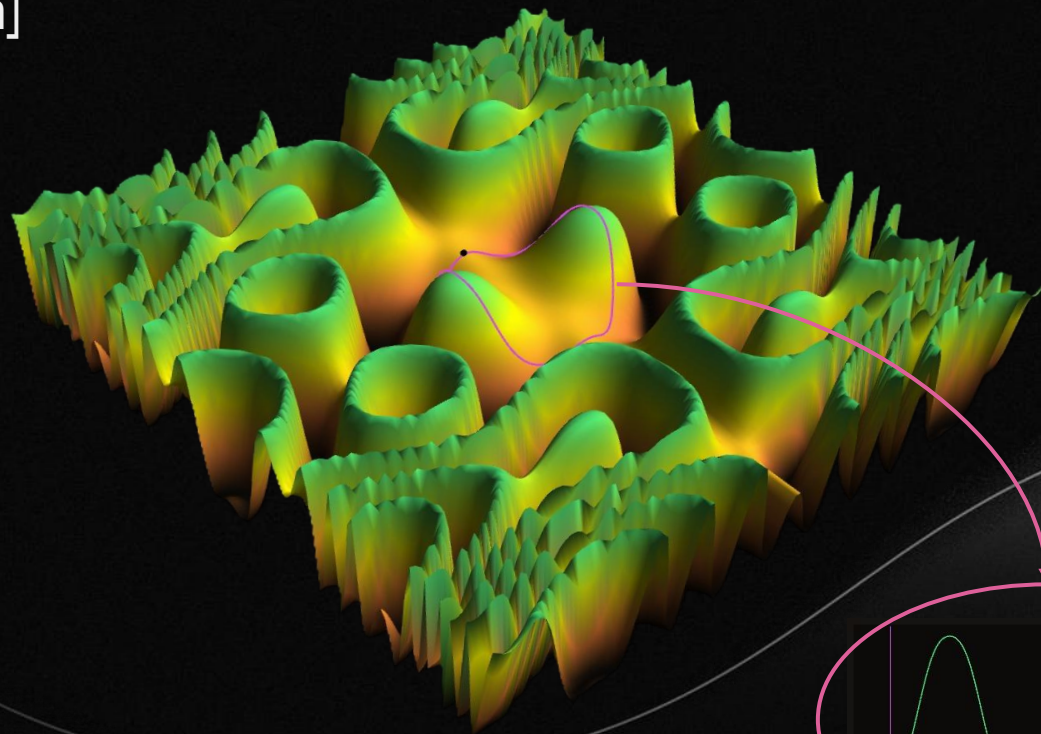
```
arduino = serial.Serial("COM3", 115200)
osc = SimpleUDPClient("127.0.0.1", 57120)

while True:
    line = arduino.readline().decode().strip()
    jx, jy, sw, ax, ay, az = map(int, line.split(","))
    osc.send_message("/sensors", [jx, jy, ax, ay, az])
    osc.send_message("/btn", sw)
```

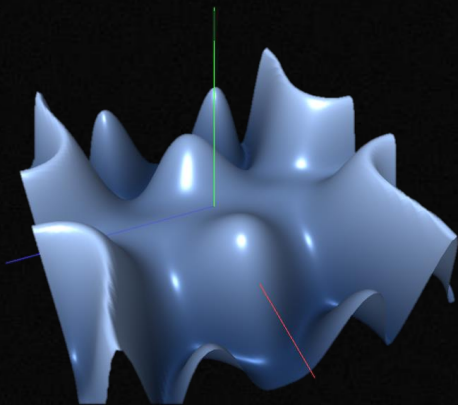
ARDUINO

SUPERCOLLIDER

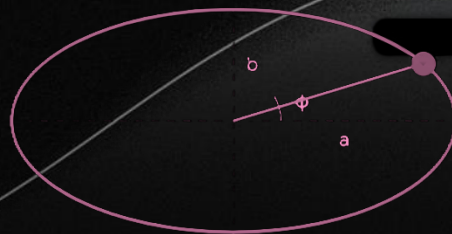
[wave terrain]




```
func: { |x, y, a, b| sin((x*sin(x) - (y*sin(y) * log(x * x + 1))) / a) },
```



```
var phi = LFSaw.ar(t[\baseFreq], -1).range(0, 2pi);  
var x = cx + (r * cos(phi));  
var y = cy + (r * sin(phi));  
  
var sig = t[\func].value(x, y, a, b) * t[\amp];
```

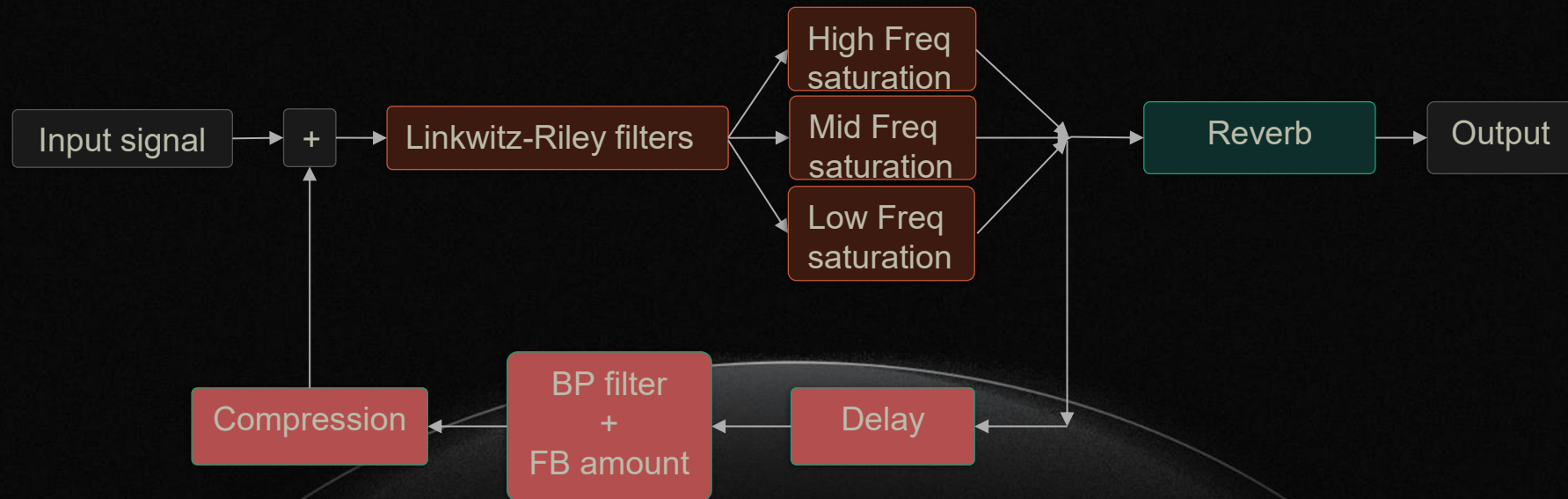




```
OSCdef(\sensorInput, { |msg|  
    var jx = msg[1].linlin(0, 1023, 0.0, 1.0);  
    var jy = msg[2].linlin(0, 1023, 0.0, 1.0);  
    var ax = msg[3].linlin(0, 1023, 0.0, 1.0);  
    var ay = msg[4].linlin(0, 1023, 0.0, 1.0);  
    var az = msg[5].linlin(0, 1023, 0.0, 1.0);  
  
    ~b_r.set(jx);  
    ~b_a.set(jy);  
    ~b_cx.set(ax);  
    ~b_cy.set(ay);  
    ~b_b.set(az);  
  
    ~procAddr.sendMsg('/sc/radius', jx);  
    ~procAddr.sendMsg('/sc/a', jy);  
    ~procAddr.sendMsg('/sc/cx', ax);  
    ~procAddr.sendMsg('/sc/cy', ay);  
}, '/sensors');
```



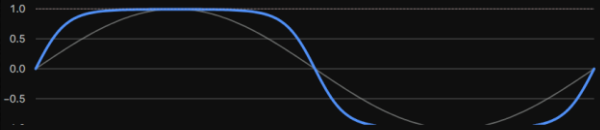
JUCE – signal flow chart



JUCE – waveshapers

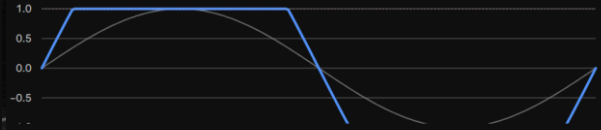
Soft Clip

```
output = std::tanh(input);
```



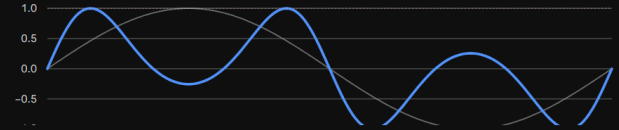
Hard Clip

```
output = juce::jlimit(-1.0f, 1.0f, input);
```



Roar Fold

```
float biased = std::tanh(input + 1.0f) - std::tanh(1.0f);  
output = std::sin(biased * 2.0f);
```

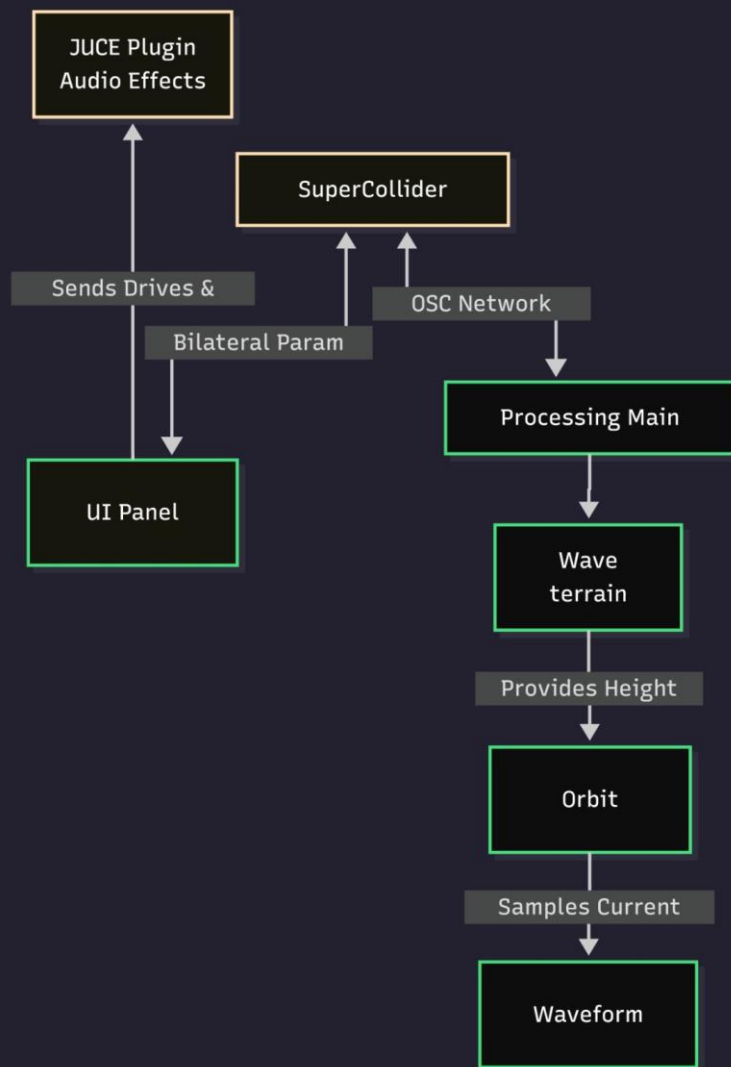


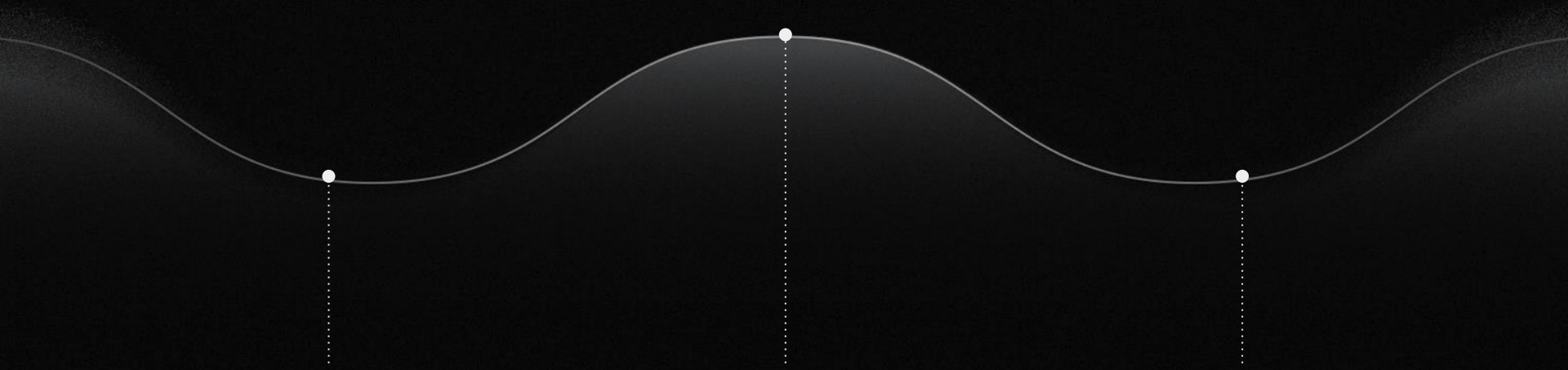
```
float autoMakeup = 1.0f / std::cbrt(drive);
```

```
channelData[sample] = output * autoMakeup;
```

Make-up gain : Channel data = output * $1/\sqrt[3]{drive}$

PROCESSING





Adding LFO/envelope
modulators for saturator
paramets

Adding, creating and saving
presets

Adding universal MIDI
interaction

FUTURE DEVELOPMENT

THANKS FOR LISTENING

